Bearing-only localization software architecture

Two independent programs exchange a measurement file; a standalone geospatial component supports both.

1. External data and ingestion

Scenario + sensor config

Synthetic object inventory, AIS-present flags, boat truth pose, range, noise, and sensor resolution.

AIS feed

Historical or live AIS records. Normalize timestamps, coordinates, vessel type, dimensions, and AtoN reports when present.

Shoreline + hydrography

Read chart / coastline geometry, project to metric coordinates, validate coverage, and rasterize land and water.

Scenario ingest feeds simulation. AIS ingest feeds search. Map ingest builds the neutral, versioned geospatial input.

2. Separate runtime programs

SIMULATOR — generates observations

`scene_builder.py` — place boat and all scene objects; preserve hidden truth
`visibility_filter.py` — apply max range and land-blocked line of sight
`angular_detector.py` — merge angularly overlapping objects; report one detection
`bearing_model.py` — compute global center bearing; add noise and uncertainty
`writers.py` — write measurements, truth, and simulated AIS catalogue separately

SEARCH — estimates boat position

`inputs.py` — read measurements, AIS catalogue, and map version
`candidates.py` — generate boat hypotheses only in water
`association.py` — match detections to AIS candidates by type and bearing
`scoring.py` — score candidate pose with bearing error and visibility
`algorithms/` — grid, RANSAC, and refinement methods as swappable solvers
`writers.py` — write estimated pose, confidence, and optional associations

EVALUATOR + BENCHMARK

`run_trials.py` — run identical cases through each search method
`metrics.py` — position error, success rate, and minimum feature count
`timing.py` — stage and total latency; CPU / GPU scaling
`report.py` — tables, figures, and reproducible result bundle

→ **measurements.json** → **estimate.json** → **evaluation**

3. Standalone map / visibility capability

land_water.py
Versioned projected raster or vector mask. Candidate generation uses water cells; visibility checks use land cells.

line_of_sight.py
Independent ray-tracing / segment-query routine. Simulator uses it to hide blocked detections; search uses it to score candidate views.

geo_cache.py
Optional tiled / cached lookup for repeated queries. Same map version and resolution are recorded in each experiment.

The visibility component is not the simulator or a search algorithm. Both programs call it through the same stable interface.

Proposed files and responsibilities

A suggested package layout. These are proposed module names, not a claim that the files already exist.

ingest/ — data adapters

`scenario.py` — load scene configuration and generate the synthetic object inventory
`ais.py` — read AIS snapshots; normalize vessel and AtoN records
`nautical_map.py` — read shoreline / hydrography sources and CRS metadata
`schemas.py` — validate inputs and versioned JSON exchange formats

geo/ — map and visibility

`project.py` — transform geographic coordinates into a local metric frame
`land_water.py` — build / query the land-water mask
`line_of_sight.py` — test whether land blocks a ray between two points
`geo_cache.py` — cache tiles and repeated visibility queries

simulator/ — measurement generation

`cli.py` — run one scene or a batch from configuration
`scene_builder.py` — place vessel, markers, and non-AIS objects; keep truth private
`visibility_filter.py` — range gate then query land visibility
`angular_detector.py` — merge close bearings; select reported class and representative target
`bearing_model.py` — center bearing, sensor noise, sigma, and apparent angular width
`writers.py` — emit measurements.json, truth.json, and simulated AIS catalogue

search/ — position estimation

`cli.py` — run search against measurement and AIS files
`inputs.py` — load reports, AIS records, and map metadata; never read truth
`candidates.py` — create coarse water-only position hypotheses
`association.py` — resolve unknown object identity and type compatibility
`scoring.py` — bearing residual, uncertainty weighting, range, and LOS consistency
`algorithms/grid.py` — exhaustive or hierarchical spatial search
`algorithms/ransac.py` — sample feature correspondences and propose poses
`algorithms/refine.py` — locally refine top candidate poses
`writers.py` — emit estimate.json and solver diagnostics

evaluation/ — scoring and benchmarks

`run_trials.py` — pair each measurement set with each search method
`metrics.py` — position error, confidence calibration, feature-count threshold
`timing.py` — per-stage runtime, full latency, and parallelism measurements
`report.py` — save CSV / JSON summaries and publication figures

schemas/ — program boundary

`observation.schema.json` — type label, global bearing, optional bearing sigma / apparent width
`truth.schema.json` — true boat position and hidden scene identity; evaluator only
`ais.schema.json` — normalized AIS object records and raw source fields
`estimate.schema.json` — estimated pose, confidence, optional associations

Data separation rule

The simulator may know every object identity, position, size, and AIS-present flag. The search input contains only reported types and bearings, the AIS catalogue, and the map. The evaluator alone joins the estimated position to simulator truth. The shared geo component supplies land / water and line-of-sight queries to both programs without coupling the simulator to the search implementation.